

ERMS

Expense Reimbursement Management System

Project Brief & Judge Q&A Prep
KSCAA Tech RRC 2026 - Group 8
Prepared July 2026

How to use this document. Every major decision below has three parts: a plain-English "explain it simply" box anyone on the team can use to open with, the actual technical reasoning (including what we didn't pick and why), and a "if a judge asks..." box with ready answers to the follow-up questions we expect. Section 7 is a one-page cheat sheet if you only have two minutes before going on stage.

1. Frontend - React + TypeScript + Vite + Tailwind CSS

Explain it simply:

Imagine five friends building a LEGO city together, each building a different building - a school, a hospital, a train station. React is everyone agreeing to use the same size LEGO bricks, so all five buildings fit together into one city instead of five mismatched piles. Vite is the delivery truck that brings you new bricks the instant you ask, instead of making you wait. TypeScript is a friend who checks your instructions before you glue anything down, so you can't accidentally build the roof upside down. Tailwind is a paint set with the colors already pre-mixed, so every building looks like it belongs in the same city.

Our choice

React for the UI, **TypeScript** for type safety, **Vite** as the build tool, **Tailwind CSS** for styling. Five role-based dashboards (Employee, Manager, Accounts, Admin, Reports) share one component library, one design-token file, and one API client (`src/lib/api.ts`).

Compared to the alternatives

Option	Why we didn't pick it (for this project)
Angular	Far more ceremony (modules, dependency injection, RxJS) than a 5-person student team building 5 screens each needs; steeper learning curve for anyone new to it mid-project.
Vue	A perfectly reasonable choice too, but smaller hiring/support pool and fewer teammates already knew it going in - React's popularity meant less onboarding time.
Plain JS / jQuery	No component reuse across 5 dashboards, no compile-time type checking - exactly the kind of codebase where the same bug (e.g. a typo'd field name) gets copy-pasted into all five modules.
Server-rendered pages (e.g. Django templates)	Would couple the UI tightly to the backend language/framework, blocking the "same language front-to-back" decision below and making five people's frontend work depend on backend availability.
Create React App instead of Vite	CRA is effectively unmaintained; Vite's dev server starts in under a second and only rebuilds the file you changed, not the whole bundle.

Plain CSS / Bootstrap instead of Tailwind	With 5 people styling 5 modules in parallel, a shared utility-class system stops every module from inventing its own slightly different shade of blue - see the Design System doc.
---	--

Q: Why React and not something "more modern" like Svelte or SolidJS?

A: Those are excellent for performance-critical apps, but our bottleneck was never rendering speed - it was five people needing to build independently without stepping on each other. React's maturity means more documentation, more Stack Overflow answers, and more teammates already comfortable with it on day one of a time-boxed competition.

Q: Is TypeScript overkill for a student project?

A: It directly caught real bugs before they shipped - a missing `@types/bcryptjs` package and a mismatched `jwt.sign()` argument type were both caught by `tsc --noEmit`, not by a user hitting an error in production.

2. Backend - Node.js + Express + TypeScript

Explain it simply:

The backend is the kitchen in a restaurant. Express is a simple, well-organized kitchen with exactly the tools you need on the counter - not a giant industrial kitchen full of machines you'll never touch. Because our kitchen (backend) and our dining room (frontend) speak the exact same language (TypeScript), the waiter doesn't need to run back and forth translating orders - food comes out faster and orders don't get mixed up.

Our choice

Node.js + Express, written in **TypeScript**, organized as one module per role (`employee`, `manager`, `accounts`, `admin`, `reports`) so each teammate owns a folder end-to-end.

Compared to the alternatives

Option	Why we didn't pick it (for this project)
--------	--

Python (Django / Flask)	Would mean two different languages across the stack. Every shared concept - what a "Claim" looks like, what fields a login response has - would need to be defined and kept in sync twice instead of once.
Java Spring Boot / .NET	Powerful and genuinely great for large enterprise teams, but heavier setup and slower iteration speed than a time-boxed competition allows - more boilerplate to write a single endpoint.
Ruby on Rails / PHP Laravel	Same "second language" problem as Python, plus a smaller pool of teammates with prior experience.
NestJS (also Node.js)	NestJS is Express's more "enterprise" cousin - heavier decorator/dependency-injection architecture. We deliberately wanted the maker-checker approval logic to be plain, explicit, and easy for a judge (or teammate) to read top-to-bottom in one file, not spread across decorators and providers.
Go / Rust	Excellent raw performance, but our workload is I/O-bound (database queries, file uploads), not CPU-bound - Node's non-blocking I/O model already fits that shape, without the added language-learning cost for a student team on a deadline.

Q: Doesn't Node.js struggle under heavy load compared to Java/Go?

A: For CPU-heavy work, yes - but an expense system's work is almost entirely "wait for the database, wait for a file to save," which is exactly what Node's event loop is built for. If this needed to crunch numbers (e.g. real-time fraud scoring across millions of rows), we'd revisit that tradeoff - see Section 8's roadmap.

Q: One language front-to-back - does that actually matter, or is it just convenience?

A: Concretely: our Claim/Employee/ApprovalStep types are defined once in the Prisma schema and Prisma generates matching TypeScript types the API imports directly - the frontend then calls that same API with full autocomplete. A field rename gets caught by the compiler on both sides at once, not by a runtime crash three modules later.

3. Storage - PostgreSQL via Prisma ORM

Explain it simply:

Storage is a filing cabinet. PostgreSQL is an extremely organized filing cabinet where every folder is connected with string to the folders it relates to - pull one employee's folder and you instantly see every claim, every manager, every payment connected to it, with nothing lost or duplicated. A NoSQL database is more like a big pile of boxes - fast to toss things into, but harder to guarantee every paper is exactly where it should be, especially when the papers represent real money. Prisma is the librarian who speaks the filing cabinet's language for us, so we never accidentally file something in the wrong drawer.

Our choice

PostgreSQL, accessed through **Prisma ORM** with one `schema.prisma` file as the single source of truth for the entire data model, shared by all five modules.

Compared to the alternatives

Option	Why we didn't pick it (for this project)
MongoDB / other NoSQL document stores	A claim's data is inherently relational - Employee -> Department -> Manager -> Claim -> Line Items -> Attachments -> Approval Steps -> Payment, all needing real referential integrity (a payment must reference a claim that genuinely exists and is genuinely verified). A document database would make us reinvent joins and integrity checks by hand in application code - exactly where subtle bugs hide.
MySQL	A reasonable relational alternative, but PostgreSQL's stricter type system (especially <code>DECIMAL</code> for money, see below) and better JSON/array support if we ever need it made it the safer default.
SQLite	Great for a single-user prototype, not for a multi-role system where five different roles are reading and writing concurrently in real time.
Firebase / Firestore	Vendor lock-in, and the same relational-integrity problem as MongoDB - plus reporting (Employee-wise / Department-wise aggregation) is much harder without real SQL <code>GROUP BY</code> support.

Raw SQL (no ORM) / other ORMs (Sequelize, TypeORM, Knex)

Prisma generates fully-typed queries directly from the schema, so a typo'd column name is a compile error, not a production bug. Its migration tooling also keeps every teammate's local database schema identical and versioned.

The one detail worth calling out: money is never a float

Every amount in the system (`totalAmount`, `approvedAmount`, payment amounts) is stored as PostgreSQL's `DECIMAL(12, 2)`, not a JavaScript number/float. Floating-point math can turn $\blacksquare 0.10 + \blacksquare 0.20$ into $\blacksquare 0.300000000000000004$ - unacceptable when the number represents real reimbursement money. The backend explicitly rounds to paise when summing line items for exactly this reason.

Q: Isn't a relational database "old-fashioned" compared to NoSQL?

A: Relational isn't old-fashioned, it's the correct shape for data that has real relationships and real integrity requirements - which financial approval data always does. NoSQL's popularity comes from problems it solves well (huge unstructured data, denormalized read-heavy feeds), which isn't the shape of this problem.

Q: What happens if two people try to approve the same claim at the same instant?

A: Every status-changing action (approve/reject/return, verify, pay) uses an atomic conditional update - it only succeeds if the claim is still in the exact expected status at that instant. A near-simultaneous second attempt gets a clean "already processed" error instead of silently double-processing the claim. This is a real, verified fix, not a theoretical answer - see `docs/judges-faq.pdf Q11`.

4. Hosting - local now, Vercel planned

Explain it simply:

Hosting is choosing where to open your lemonade stand. Right now our stand is set up in our own backyard (running on our own laptops) so we can practice and get every detail right without anyone watching. Vercel is a great spot in the middle of town - easy for anyone in the world to walk up to, and someone else takes care of paving the road and fixing the streetlights (the servers) for us, so we just focus on selling lemonade.

Current state (today, for this competition)

ERMS runs locally via a one-click launcher (`run.bat`) that checks whether Node.js and PostgreSQL are installed, installs whichever is missing, sets up the database, and starts both servers - deliberately built so any teammate or judge can run the exact same system on their own machine in one step, with no shared server to keep online or pay for during judging.

Planned: Vercel

Piece	Plan on Vercel
Frontend (React/Vite build)	Deployed directly as a static site on Vercel's CDN - this is exactly what Vercel is built for: zero-config deploys straight from GitHub, automatic HTTPS, a preview URL for every commit/PR.
Backend (Express API)	Needs adapting: Vercel runs code as short-lived serverless functions rather than one long-running Express process, so each API route becomes its own function (or the Express app is wrapped to run inside one). This is a real, scoped piece of migration work we've identified but not yet done.
Database (PostgreSQL)	Vercel doesn't host Postgres itself - we'd move to a managed serverless-friendly Postgres (e.g. Vercel Postgres/Neon, or Supabase) with connection pooling, since serverless functions open many short-lived connections rather than one persistent one like our current setup.

Why Vercel specifically

- Free tier is generous enough for a student/hackathon project with no ongoing hosting budget.
- Deploys straight from our GitHub repo - every push can automatically get a live URL.

- Global CDN means judges anywhere get a fast first load, not just people near our server.
- Preview deployments per branch/PR make it easy to show a reviewer a specific change without touching the live demo.

Q: Why isn't it deployed on Vercel already, if that's the plan?

A: We prioritized building a fully working, security-hardened, tested system first - found and fixed real bugs (path traversal, a broken self-approval check, an unenforced duplicate-flag) through actually attacking our own app, not just reading the code. Deploying to a genuinely different runtime (serverless functions instead of a long-running server) is real migration work, and we didn't want to rush it days before judging and risk a production-only bug appearing live in front of judges. It's next on our roadmap, not forgotten.

Q: Could you deploy the whole thing as-is, Express server and all?

A: Not cleanly on Vercel specifically - Vercel's model is serverless functions, not a persistent Express process. We could deploy the Express API as-is on a platform built for long-running servers (Render, Railway, a small VM) while still putting the frontend on Vercel - a hybrid approach and a legitimate alternative to full serverless migration.

5. UI & UX

Explain it simply:

UI is what a playground looks like - bright colors, clear signs. UX is whether the playground is actually fun and easy to use - are the swings where you'd expect them? Do you know which slide is for big kids and which is for little kids? We made sure every person (Employee, Manager, Accounts, Admin, CEO) only sees the playground equipment meant for them, so nobody gets confused pressing a button that isn't theirs.

Our approach

- **Role-based dashboards.** Each of the 5 roles sees only the navigation and screens relevant to their job - an Employee never sees Payment Processing, an Accounts user never sees Employee Management. Less to look at, fewer mistakes.
- **One shared design system** (colors, typography, button/card styles) documented in `docs/design-system.md` so five independently-built modules still look like one product, not five different apps stitched together.

- **Consistent decision pattern.** Manager's Approve/Return/Reject and Accounts' Verify/Reject screens use the identical three-button-plus-required-remarks layout, so learning one screen means already knowing the other.
- **Color-coded status badges** (pending = amber, approved = emerald, rejected = red, returned = orange, paid = blue) applied consistently everywhere a claim's status appears, so a user recognizes a claim's state at a glance without reading text.
- **Errors prevented, not just reported.** The password-strength checklist updates live as you type, showing exactly which rule is still unmet; a claim genuinely cannot be submitted without a bill attached, both in the form and enforced again on the server.
- **Iterated from real usage, not just a mockup.** Several UI details exist because live testing surfaced a real problem: the category field became a dropdown (was a raw ID a user had to type), attachment preview was fixed for managers, and CSV export was fixed across every report screen.

Q: How did you validate the UX actually works, not just that it compiles?

A: By using it end-to-end as each role, repeatedly, including two dedicated QA passes where we deliberately tried to break it like a real user or attacker would - not just reading the code. That's how we found the category-dropdown usability gap and the broken attachment preview, alongside the security bugs mentioned elsewhere in this brief.

6. Other things worth having ready

Security & trust

- **Maker-checker enforced server-side, not just hidden in the UI** - an employee's claim can only be approved by their own specifically assigned manager, checked on every request, not just in the screen that lists pending claims. The one exception is the CEO, who has no manager above them and is the only role allowed to approve their own claim.
- **Append-only audit trail** - every claim action, login, and password change writes exactly one row through a single shared helper; there is no update or delete route for it anywhere in the system.
- **Passwords** are bcrypt-hashed, never logged or stored in plain text, and must meet a strength policy enforced identically everywhere a password is set.
- **Session idle timeout** is tracked against real request activity in the database, not a fixed timer from login.
- We found and fixed a real **path traversal vulnerability** in bill uploads during our own testing (a crafted claim ID could have written a file outside the app entirely) - a good story for "how seriously do you take security," since it shows we test for it rather than assume it away.

How we know it actually works

Every major feature in this system was exercised end-to-end against a running instance with real HTTP requests and, where relevant, in the live browser UI - not just unit-level assertions. Two full rounds of dedicated QA testing (playing all five roles, deliberately trying to break things) found and fixed real bugs: a duplicate-flag check that could be silently skipped, a session timeout that never actually reset, and the self-approval gap and path traversal issue above. When we added partial manager approval and real notifications, we verified the entire chain live - submit, partially approve, verify, pay - before considering either feature done.

What's deliberately not built yet (say this proactively, don't wait to be asked)

- **Multi-level approval by amount** - the Approval Matrix data model and admin screen already support department/amount-based escalation rules, but the live approval flow is currently always exactly one manager step. We chose not to bolt this onto an approval path we'd just finished hardening for correctness under concurrent access.
- **Excel/PDF report export** - CSV export works everywhere reports appear; Excel/PDF are on the roadmap.
- **MFA enrollment** - the schema already has the fields (`mfaSecret`, `mfaEnabled`); the enrollment flow isn't wired up yet.
- **Email delivery of notifications** - notifications exist and are created correctly in-app (see below); emailing them out is a natural next step once a real email service is wired in.
- **AI roadmap** - OCR-based receipt data extraction and ML-based fraud scoring, both designed to augment (not replace) today's deterministic duplicate check, which stays as the auditable compliance

baseline.

Being upfront about this list is deliberate: judges respect a team that knows exactly where its own edges are far more than a team that overclaims and gets caught out in Q&A.

Two features judges may not know to ask about but are genuinely new

- **Real in-app notifications.** The assigned manager is notified the moment a claim is submitted or resubmitted; the employee is notified on every manager decision, every Accounts decision, and on payment.
- **Partial approval.** A manager isn't limited to a binary approve/reject/return - on Approve, they can authorize less than the full claimed amount (e.g. one line item is outside policy). Accounts then verifies and pays exactly that approved amount, and the split is visible everywhere the claim's amount appears, to both the manager and the employee.

7. Two-minute cheat sheet

If you only have time to glance at one page before going up, this is it.

Why React/Node/Postgres and not X?

One language (TypeScript) front-to-back for a 5-person team splitting work in parallel; Postgres because claim data is genuinely relational and financial data needs real transactions and exact decimal math, not floats.

Why isn't it live on Vercel yet?

Frontend deploys to Vercel as-is; the Express backend needs real migration work to serverless functions (or a hybrid host), which we scoped deliberately rather than rush before judging.

How do you stop someone approving their own claim?

Every approval checks the actual manager-employee relationship in the database, on every request - not department membership, not role alone. CEO is the sole, structural exception.

Can a manager approve part of a claim?

Yes - partial approval is real and live; Accounts verifies and pays the approved amount, not the original claimed total, and both the manager and employee see the split.

Does anyone get told when something happens to their claim?

Yes - the manager is notified on submission/resubmission, the employee on every decision and on payment, all visible in-app on a Notifications screen.

How do you know it actually works, not just compiles?

Two full rounds of live QA, deliberately trying to break our own system, which is exactly how we found and fixed a path traversal bug and a broken self-approval check - not something we're guessing about.

What would you build next?

In order: deploy to Vercel, live multi-level approval routing, Excel/PDF export, MFA enrollment, emailing the notifications that already exist in-app.